Academic Year : 2026-2027

Dr. Vasantraodada Patil Shetkari Shikshan Mandal's Padmabhushan

Vasantraodada Patil Institute of Technology

Department of Computer Science Engineering

CERTIFICATE

This is to certify that the project synopsis entitled

WAYFINDER - CAMPUS NAVIGATION GUIDE

is submitted by the following students

NAME	PRN NO	ROLL NO

Incharge	H.O.D	Principal
Mrs. P.S. Patil	Mr. A.B. Jagadale	Dr. S.S. Mohite

WAYFINDER AT A GLANCE

Synopsis Overview

Abstract

Finding a particular building on a college campus can be difficult for newly admitted students, parents, faculty members, and other visitors. A campus may contain academic blocks, hostels, offices, food facilities, recreation areas, and multiple pedestrian paths, while signs and informal directions may be incomplete or unfamiliar to a first-time visitor. WayFinder proposes a lightweight web application that brings campus place discovery and walking directions into one interface.

The application combines a React single-page frontend with a FastAPI backend and an SQLite data store. Users can search and filter a directory of campus locations, view markers on an OpenStreetMap-based Leaflet map, inspect location details, and select a source and destination. For navigation, the backend represents walkable paths as a weighted graph. It maps selected coordinates to nearby graph nodes, applies Dijkstra's shortest-path algorithm using path distance as the edge weight, and returns route coordinates, total estimated distance, estimated walking time, and simple turn instructions. Contact-form submissions are stored in the database.

This synopsis describes the current student-project prototype and its development method. The sample graph and coordinates are illustrative campus data and must be checked against a real campus survey before operational use. Walking time is an estimate based on an assumed speed, and the prototype does not claim live positioning, indoor navigation, accessibility validation, or a surveyed pedestrian network. Expected outcomes are therefore functional demonstration outcomes, not measured user-study results.

Keywords

Campus wayfinding; web mapping; OpenStreetMap; React; FastAPI; SQLite; Dijkstra's algorithm; pedestrian routing.

Contents

- 1. Introduction
- 2. Literature Survey
- 3. Problem Statement
- 4. Objectives
- 5. Methodology
- 6. Proposed System
- 7. Future Scope
- 8. Conclusion
- 9. References

CONTEXT AND MOTIVATION

1. Introduction

1.1 Background

A college campus functions like a small, familiar-to-regulars but unfamiliar-to-visitors environment. Important destinations are distributed across a network of paths and buildings. A person may know the name of a department or service but not its location, the nearest entrance, or the route that avoids unnecessary walking. This creates friction during admissions, examinations, events, appointments, and the first weeks of a new academic term.

Digital maps can make spatial information easier to access, but general-purpose maps do not always expose the campus-specific directory and path knowledge a visitor needs. A useful campus guide should connect a searchable place catalogue with a map and make the next action understandable. In this project, a location is represented by its name, category, coordinates, description, image reference, and opening-hours text. The map provides spatial context, while route instructions translate a computed path into short phrases such as “Go straight for … metres” or “Turn left at …”.

1.2 Project concept

WayFinder is a responsive web application for discovering and navigating to selected campus places. The homepage introduces the service and highlights popular locations. Search and category filters narrow the place list; selecting a result opens its details or focuses it on the map. The navigation interface accepts a source and destination, requests a route from the backend, and visualises the returned coordinate sequence as a polyline. Distance and walking-time estimates accompany the route instructions.

The technical design separates the interface, API, routing logic, and stored data. React, React Router, Axios, and React Leaflet form the browser interface. FastAPI exposes REST endpoints, SQLAlchemy maps application records to SQLite tables, and Leaflet displays map layers and route geometry. OpenStreetMap supplies the displayed basemap, subject to its attribution and usage requirements [7], [8].

1.3 Importance and scope

The project is important as a practical demonstration of how web development, databases, mapping, graph theory, and human-centred information design can work together. It offers visitors one starting point for finding campus facilities and demonstrates a graph-based approach to walking directions. The current version is a prototype for a single sample campus: location coordinates and path edges need local verification. It is not a replacement for emergency signage, campus security instructions, or a professionally surveyed accessible routing service.

PRIOR WORK AND EXISTING APPROACHES

2. Literature Survey

2.1 Digital campus wayfinding

University wayfinding research spans online directories, mobile navigation, indoor mapping, and context-aware systems. A campus application must represent both destinations and the connections between them. A directory answers “what is here?”; a map answers “where is it?”; a routing service answers “how can I get there?” These functions are related but require different data and interaction choices.

2.2 Event-driven campus navigation

The IEEE Xplore work titled “An event-driven university campus navigation system on android platform” describes a mobile approach to campus navigation [2]. Its mobile-platform framing is relevant because visitors often need directions while moving around campus. Compared with a native mobile application, WayFinder uses a browser-based interface that can be opened from a shared link without first installing an app. The trade-off is that a web prototype has fewer built-in device capabilities and does not automatically provide reliable live-position tracking.

2.3 Augmented-reality and indoor navigation

An IEEE study on an augmented-reality-based indoor positioning navigation tool explores image-processing and augmented-reality guidance for a building environment [3]. Such techniques can provide visual cues when satellite positioning is weak indoors, but they require device camera interaction and carefully prepared indoor references. WayFinder focuses first on a simpler two-dimensional outdoor campus map, which is easier to explain and maintain for a small student prototype. Indoor room-level guidance remains a potential extension rather than a current capability.

2.4 Accessibility-oriented guidance

GuideBeacon investigates beacon-based indoor wayfinding for people who are blind, visually impaired, or disoriented [4]. Another campus-navigation study addresses blind, hearing-impaired, and visually impaired users in the ITU Ayazaga campus context [5]. These works highlight that navigation is not only a geometry problem: guidance must be perceivable, safe, and appropriate to the user. WayFinder’s current text directions and visual map form a baseline, but they have not been validated with assistive-technology users and do not yet model curb ramps, stairs, surface quality, or accessible entrances.

2.5 Implication for the project

The literature suggests a sensible staged design: begin with reliable place data and outdoor paths, return explainable directions, and treat accessibility and indoor positioning as explicit data and testing requirements. This project adopts that bounded first step, while documenting what a future campus survey and user evaluation would need to add.

COMPARISON AND RESEARCH GAP

2. Literature Survey Comparison

Approach / reference	Strength reported or implied	Design implication for WayFinder
Mobile, event-driven campus navigation [2]	Campus-focused mobile access and navigation workflow	Offer campus-specific POIs in a browser-first interface; avoid claiming native mobile features
AR indoor positioning [3]	Visual, context-aware indoor guidance	Keep first release to mapped outdoor paths; indoor guidance needs a separate positioning method
GuideBeacon indoor wayfinding [4]	Beacon-based assistance for users who can become disoriented	Treat accessible instructions and user testing as important future work
Campus multimodal navigation [5]	Considers differing sensory needs in a campus context	Do not equate a visible route line with inclusive navigation
3D university mapping with NavMesh/A* [6]	Detailed spatial representation and indoor route planning	A 2D graph is simpler and explainable for an initial prototype; more detailed maps add data and maintenance cost
MazeMap campus study [7]	Examines deployed indoor mapping and mobility in university settings	Campus mapping benefits from maintained, institution-specific spatial data

2.6 Research gap and project position

Many advanced systems rely on technologies such as indoor positioning, beacons, augmented reality, 3D models, or detailed institutional datasets. These approaches can be powerful, but they require additional infrastructure, data collection, device support, and ongoing maintenance. A small academic prototype benefits from a transparent, low-cost baseline that can be demonstrated and incrementally improved.

WayFinder's contribution is an integrated and explainable prototype: a structured place catalogue, name/category search, interactive map, campus walking graph, Dijkstra shortest-distance computation, basic text directions, and contact-message storage. Its main gap is also clear: the sample graph is not yet an authoritative pedestrian network. A real deployment would require campus validation, route-safety review, accessible-path attributes, and testing with intended users. The project therefore demonstrates a method and working architecture, not a claim of superiority over the specialist systems reviewed.

Review basis The comparison summarises the cited systems at a high level. The synopsis uses publisher and project documentation for factual details; full-text access and campus-specific validation should be completed before formal research claims are made.

NEED, SCOPE, AND ASSUMPTIONS

3. Problem Statement

3.1 Problem definition

New students, parents, faculty members, and visitors may not know where important campus facilities are located or how to walk between them. Information may be distributed across signs, printed maps, websites, or informal directions. When a visitor knows a destination name but not its location, the lack of a unified, searchable, map-based guide can cause confusion, repeated enquiries, and avoidable walking.

The project addresses the following problem: how can a visitor use a simple web interface to find a named campus place and receive an understandable walking route between two selected campus points, using a maintainable dataset and a transparent shortest-path method?

3.2 Specific difficulties

- Visitors may not know the official name, category, or position of a facility.
- A list of destinations without map context does not show spatial relationships.
- A straight line between two coordinates may cross buildings or areas without a pedestrian path.
- A route line alone does not communicate distance, approximate time, or the next turn.
- Location details and contact feedback need a simple data-backed interface.

3.3 Scope

The prototype includes a campus location catalogue, category and text search, map markers and popups, location detail pages, a graph-based walking route response, estimated distance/time, written turn guidance, toast feedback, and a contact form stored through the backend. The browser uses React and Leaflet; the service layer uses FastAPI and SQLAlchemy; SQLite is used for the sample deployment.

3.4 Assumptions and limitations

The route graph contains a finite set of manually supplied pedestrian junctions and edges. Each edge is weighted by geographic distance between its endpoints. Selected source and destination points are attached to their nearest graph nodes; the current implementation rejects a point more than 500 metres from the mapped graph. The algorithm minimises total distance within this graph; it cannot find a path that was omitted from the graph. Walking time is estimated using an assumed rate of 80 metres per minute and rounded up to whole minutes. Neither the rate nor the graph has been validated by a campus-wide field study. The prototype does not track the user's live location, reroute dynamically, or guarantee step-free or safe access.

Engineering interpretation Dijkstra returns the shortest path only relative to the graph and edge weights supplied. Accurate directions therefore depend first on accurate campus paths, coordinates, and route attributes.

WHAT THE PROJECT INTENDS TO ACHIEVE

4. Objectives

4.1 Primary objective

To design and implement a responsive web-based campus navigation guide that helps a user discover campus locations and obtain an understandable shortest-distance walking route between selected points on a mapped pedestrian graph.

4.2 Specific objectives

1. Create a structured location catalogue containing each place's name, category, latitude, longitude, description, image reference, and opening-hours text.
2. Provide a simple responsive interface with navigation between Home, Map, Location Details, About, and Contact views, including a custom not-found page.
3. Allow users to search location names, filter by category, and inspect matching places through cards, details, and map markers.
4. Display campus places on an OpenStreetMap-based Leaflet map with marker popups and route geometry.
5. Represent pedestrian paths as a weighted graph and apply Dijkstra's algorithm to select the minimum-distance path between the nearest connected access nodes.
6. Return a route polyline, distance estimate, walking-time estimate, and simple turn-by-turn text from the route API.
7. Validate API input and provide clear success/error feedback; store submitted contact messages in SQLite.
8. Keep the implementation modular and explainable so it can be demonstrated, tested, and extended by a student team.

4.3 Demonstration acceptance checks

The project can be demonstrated against observable checks rather than invented performance results: (i) a known location can be found by name or category; (ii) selecting a marker shows the location identity; (iii) a connected source/destination pair returns a route whose map line follows graph nodes rather than a single straight segment; (iv) the response includes non-negative distance and a positive estimated time; (v) directions begin and end with clear guidance; and (vi) a valid contact form submission appears in the database. These checks confirm functional behaviour, not route accuracy or user satisfaction.

4.4 Expected academic value

The work provides an applied exercise in full-stack development, REST API design, relational data modelling, map visualisation, graph representation, shortest-path algorithms, input validation, and technical documentation. It also creates a concrete foundation for later evaluation with real campus paths and users.

DEVELOPMENT PROCESS AND FLOWCHART

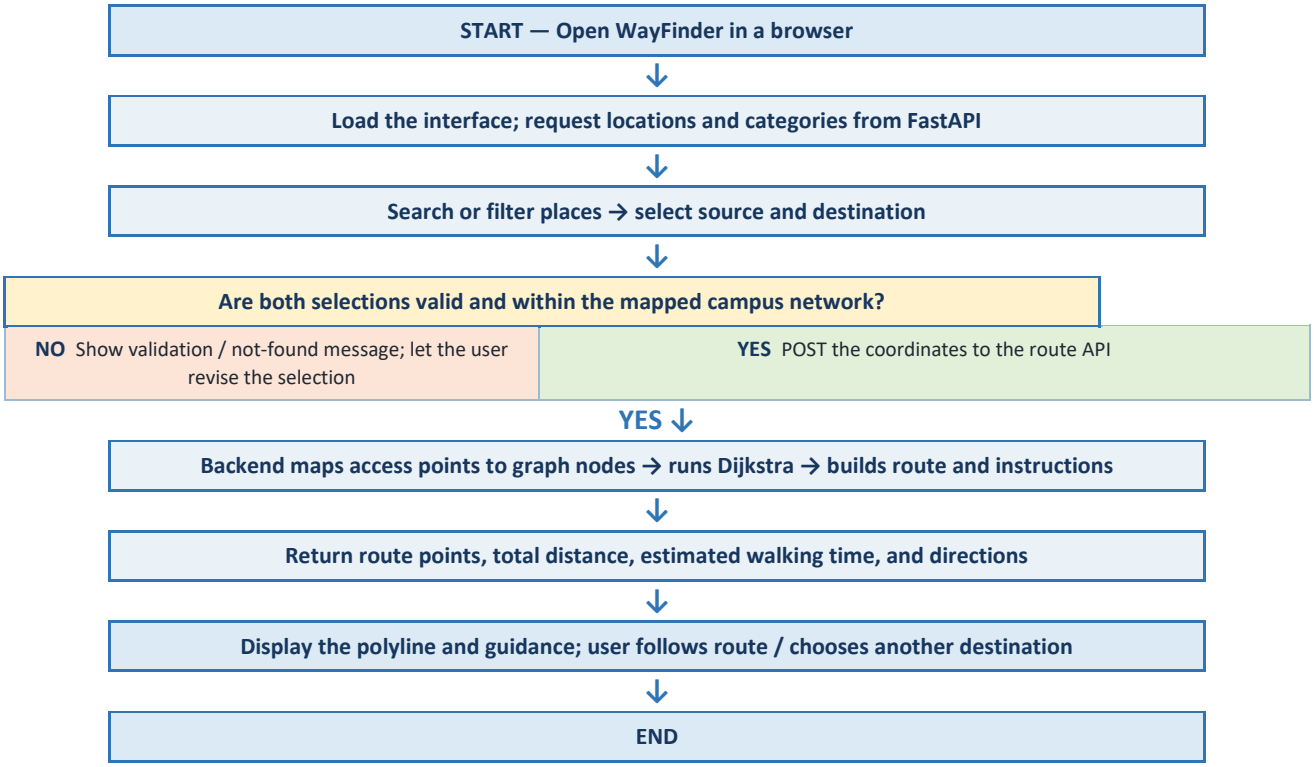
5. Methodology

5.1 Development method

The project follows an incremental, prototype-oriented development process. Requirements are translated into user tasks, the data model and route graph are designed, frontend and backend modules are implemented, and the integrated system is checked against representative use cases. Separating the place catalogue from the routing graph allows the project to maintain location records and path topology as related but distinct data.

5.2 Application workflow

The following flowchart summarises the normal user journey and API interaction. It is an editable Word table diagram and represents the current prototype workflow.



5.3 Implementation stages

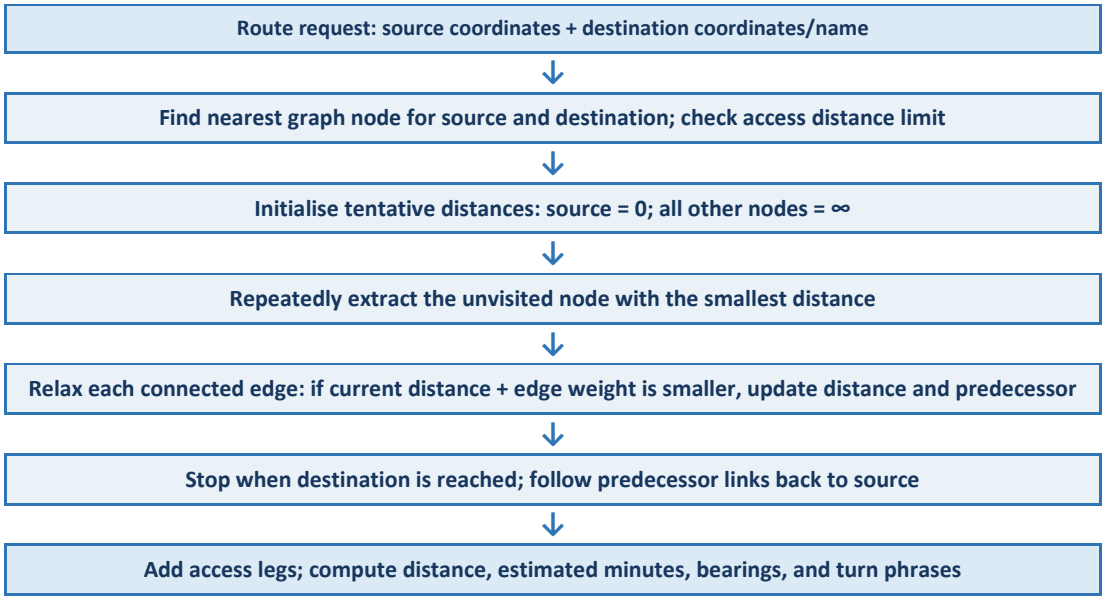
The staged workflow is: (1) identify user and system requirements; (2) prepare sample places and pedestrian connections; (3) define SQLAlchemy entities and REST schemas; (4) implement search, location, contact, and route endpoints; (5) build reusable React views and map components; (6) connect the client through Axios; and (7) perform functional and integration checks. Location labels, coordinates, images, and opening hours are sample data and should be reviewed by the institution before publication.

DIJKSTRA ROUTING, TURN GUIDANCE, AND VERIFICATION

5. Methodology (continued)

5.4 Route computation

The pedestrian network is modeled as a weighted graph G = (V, E). Each junction or path waypoint is a vertex v ∈ V. Each walkable connection is an edge e ∈ E. The current prototype treats connections as undirected; the edge cost is the Haversine distance between endpoint coordinates in metres. The route request supplies source and destination coordinates. The system attaches each to its nearest graph node, calculates the shortest graph path, and combines that path with the source and destination access legs.



5.5 Direction generation

After the path is recovered, consecutive coordinates provide segment bearings and segment lengths. The implementation compares each pair of bearings: a small change is described as continuing straight, a left or right change becomes a turn instruction, and a large reversal is treated as a U-turn. Segment lengths are rounded to metres for display. A final arrival message names the destination. These are geometric instructions generated from the modeled path, not a surveyed description of landmarks or a live navigation voice prompt.

5.6 Testing and evaluation plan

Functional testing should cover known locations, no-match search, category filtering, invalid route input, route requests for connected nodes, disconnected or out-of-network selections, and contact form validation. Integration testing should verify that the browser sends the expected JSON shapes and presents API errors clearly. Route verification should compare returned path edges with the maintained campus graph and manually walk representative routes after the graph is surveyed. Performance can be measured for API response time and page load on the target network; this synopsis reports no measurements because no benchmark dataset or test results were supplied.

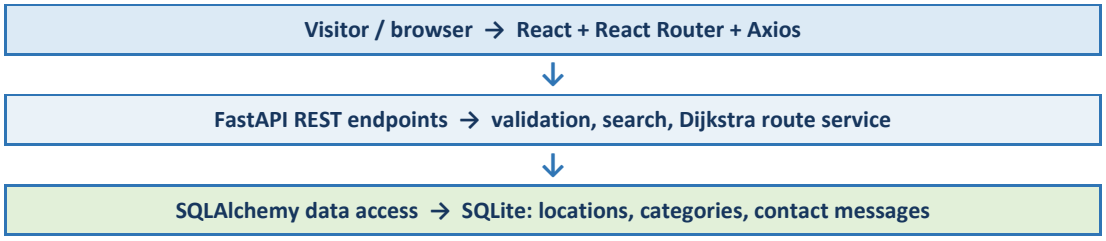
Key method dependency Shortest-path correctness can be checked computationally against the graph. Whether that path is a safe and usable real-world walking route requires ground-truth campus data and field validation.

ARCHITECTURE, MODULES, AND DATA

6. Proposed System

6.1 System overview

WayFinder is organised as a browser client, REST backend, relational store, and map-tile provider. The React application manages page state and user interactions. Axios sends HTTP requests to FastAPI. FastAPI validates request and response shapes, calls route/search logic, and uses SQLAlchemy sessions to read or write SQLite records. React Leaflet renders the map and route geometry using Leaflet; OpenStreetMap provides the underlying tiles with required attribution [7], [8].



6.2 Main modules

Home and discovery. Search bar, popular locations, category cards, map preview, and shared navigation.

Map and location detail. Leaflet markers and popups; selected location name, category, description, image, hours, coordinates, and a navigation action.

Route service. Accepts source/destination coordinates and optional destination label; returns route coordinate pairs, distance in metres, estimated walking minutes, and direction strings.

Search and category service. Searches the location catalogue and lists category options so the client can filter results.

Contact service. Validates and stores a visitor's name, email, message, and creation time in SQLite.

Feedback and error handling. Toast notifications communicate success, no results, route errors, and contact submission status.

6.3 Data entities and APIs

The database contains Category (identifier and unique name), Location (identifier, name, category, latitude, longitude, description, image path, and opening-hours text), and ContactMessage (identifier, name, email, message, and creation timestamp). The API surface includes GET /health; GET /locations and GET /locations/{id}; GET /categories; POST /search; POST /route; and POST /contact. Standard HTTP status codes and request validation communicate missing records, invalid data, and successful operations. The service is intentionally small enough to explain in a student presentation while keeping route calculation separate from interface rendering.

Map-data note OpenStreetMap is the basemap, not the source of the prototype's campus routing graph. The walking edges and sample place coordinates need to be verified independently and updated with permission and attribution where applicable.

RESPONSIBLE NEXT STEPS

7. Future Scope

7.1 Possible improvements

Ground-truth campus mapping. Survey paths and entrances, verify coordinates, add missing pedestrian links, and appoint a data owner to maintain changes.

Accessible routing. Add attributes such as steps, ramps, surface, slope, width, and accessible entrances; allow users to select route preferences and validate routes with people with disabilities.

Live position and rerouting. With user consent, evaluate browser geolocation, position accuracy, battery use, privacy controls, and route recalculation when a visitor leaves the planned path.

Indoor navigation. Model building floors and room-level routes; assess QR codes, Wi-Fi, Bluetooth beacons, or visual positioning against cost, privacy, and maintenance requirements.

Richer routing costs. Support one-way paths, closures, construction, safety constraints, lighting, stairs, and time-dependent conditions; minimise a user-selected cost rather than distance alone.

Evaluation and deployment. Conduct usability tests with students and visitors, measure search and API response times under stated conditions, add automated tests, back up data, and monitor a hosted production service.

Multilingual and assistive output. Offer local-language labels, screen-reader-structured instructions, larger text, high contrast, and optional audio prompts after accessibility evaluation.

8. Conclusion

WayFinder addresses a practical campus problem: visitors may know where they need to go by name but lack a convenient way to locate it and understand a walking route. The proposed system integrates a searchable campus directory, map-based place browsing, a FastAPI service, SQLite records, and graph-based route calculation in a responsive web application. Dijkstra's algorithm is used to minimise distance across the modeled pedestrian network, while segment bearings support simple written turn instructions and a distance-based time estimate.

The project is suitable as an initial engineering prototype because its key elements are visible and explainable: the data model, API request, campus graph, shortest-path algorithm, and map display. Its results depend on the quality of the campus data, and its walking-time and turn text are estimates rather than field-validated guidance. The next responsible step is to validate the graph with the institution, evaluate routes with intended users, and add accessibility and privacy requirements before real-world deployment. Within those limits, WayFinder provides a clear foundation for a useful campus navigation service and a strong platform for further academic development.

RESEARCH PAPERS, STANDARDS, AND OFFICIAL DOCUMENTATION

9. References

[1] E. W. Dijkstra, “A note on two problems in connexion with graphs,” Numerische Mathematik, vol. 1, pp. 269–271, 1959. doi: 10.1007/BF01386390. https://doi.org/10.1007/BF01386390

[2] “An event-driven university campus navigation system on android platform,” IEEE Xplore, document 7083850. https://ieeexplore.ieee.org/document/7083850/

[3] “Augmented reality based indoor positioning navigation tool,” in 2011 IEEE Conference on Open Systems (ICOS), 2011. doi: 10.1109/ICOS.2011.6079276. https://doi.org/10.1109/ICOS.2011.6079276

[4] “GuideBeacon: Beacon-based indoor wayfinding for the blind, visually impaired, and disoriented,” in 2017 IEEE International Conference on Pervasive Computing and Communications (PerCom), 2017. doi: 10.1109/PERCOM.2017.7917858. https://doi.org/10.1109/PERCOM.2017.7917858

[5] “Navigation system for blind, hearing and visually impaired people in ITU Ayazaga campus,” in 2017 International Conference on Computer Science and Engineering (UBMK/ICCNI), 2017. doi: 10.1109/ICCNI.2017.8123814. https://doi.org/10.1109/ICCNI.2017.8123814

[6] “3D Mapping and Indoor Navigation for an Indoor Environment of the University Campus,” in 2023 IEEE International Symposium on Wireless Technology & Applications (ISWTA), 2023. doi: 10.1109/ISWTA58588.2023.10249749. https://doi.org/10.1109/ISWTA58588.2023.10249749

[7] “Navigating MazeMap: indoor human mobility, spatio-logical ties and future potential,” arXiv:1401.5297, 2014. https://arxiv.org/abs/1401.5297

[8] OpenStreetMap Foundation, “Copyright and License.” https://www.openstreetmap.org/copyright (accessed 16 Oct. 2026).

[9] Leaflet, “Leaflet Documentation.” https://leafletjs.com/reference.html (accessed 16 Oct. 2026).

[10] FastAPI, “FastAPI Documentation.” https://fastapi.tiangolo.com/ (accessed 16 Oct. 2026).

[11] SQLAlchemy, “SQLAlchemy ORM Documentation.” https://docs.sqlalchemy.org/ (accessed 16 Oct. 2026).

[12] React, “React Documentation.” https://react.dev/ (accessed 16 Oct. 2026).

[13] SQLite, “SQLite Documentation.” https://www.sqlite.org/docs.html (accessed 16 Oct. 2026).

Citation note Numbered references are cited in the Literature Survey and system description. IEEE Xplore entries are listed by their discoverable publication title and DOI/document record; verify full author, venue, and pagination details through the publisher record before final institutional submission.